

Reading Assignment: Please answer three questions below:

- What are the advantages of Polymorphism?
 - Có thể tái sử dụng mã: giảm lượng code trùng lặp bằng cách sử dụng cùng 1 phương thức hoặc interface cho nhiều loại đối tượng khác nhau (cùng 1 lớp cha).
 - Các đối tượng thuộc các lớp khác nhau có thể được xử lý như cùng một kiểu (như trong bài: CD, DVD, Book đều là Media).
 - Dễ thay đổi, nâng cấp code: khi muốn thay đổi một phương thức lớp Media mới, không cần sửa hết các lớp con.
 - Không cần viết nhiều hàm riêng cho từng loại đối tượng, chỉ cần gọi chung một phương thức.
- How is Inheritance useful to achieve Polymorphism in Java?
 - Các lớp con (Book,DVD,CD) kế thừa thuộc tính và phương thức từ lớp cha Media). Kế thừa cho phép các lớp con override phương thức của lớp cha, từ đó cùng một phương thức có thể có nhiều cách triển khai khác nhau.
- What are the differences between Polymorphism and Inheritance in Java?
 - Kế thừa: dùng để tái sử dụng code, tập trung vào cấu trúc (lớp con kế thừa thuộc tính, phương thức lớp cha). Đơn kế thừa và đa kế thừa.
 - Đa hình: dùng để thay đổi hành vi một cách linh hoạt, tập trung vào hành vi (cùng phương thức, khác cách thực thi). Ghi đè và nạp chồng.

⇒ Kế thừa là cơ chế để xây dựng mối quan hệ phân cấp giữa các lớp, trong khi Đa hình sử dụng mối quan hệ đó để thực hiện các hành vi linh hoạt.

17. Sort media in the cart

- What class should implement the Comparable interface?
 - Lớp Media nên implement Comparable<Media>, vì tất cả các loại media (Book, DVD, CD) đều kế thừa từ Media.
- In those classes, how should you implement the `compareTo()` method to reflect the ordering that we want?
 - Sắp xếp theo title rồi cost:

```
53 @Override
54 public int compareTo(Media other) {
55     int titleCompare = this.title.compareToIgnoreCase(other.title);
56     if (titleCompare != 0) return titleCompare;
57     return Float.compare(this.cost, other.cost);
58 }
```

- Sắp xếp theo cost rồi title:

```

53 @Override
54 public int compareTo(Media other) {
55     int costCompare = Float.compare(this.cost, other.cost);
56     if (costCompare != 0) return costCompare;
57     return this.title.compareToIgnoreCase(other.title);
58 }

```

- Can we have two ordering rules of the item (by title then cost and by cost then title) if we use this Comparable interface approach?
 - Không vì Comparable chỉ cho phép một quy tắc sắp xếp mặc định duy nhất (dùng compareTo()).
 - Nếu muốn nhiều cách sắp xếp khác nhau (như theo tiêu đề, theo giá, theo độ dài,...) phải dùng Comparator.
- Suppose the DVDs have a different ordering rule from the other media types, that is by title, then decreasing length, then cost. How would you modify your code to allow this?
 - Ta có thể override lại phương thức compareTo() trong class DigitalVideoDisc theo title-length-cost. Các lớp khác vẫn dùng compareTo() của Media (theo cost, title)

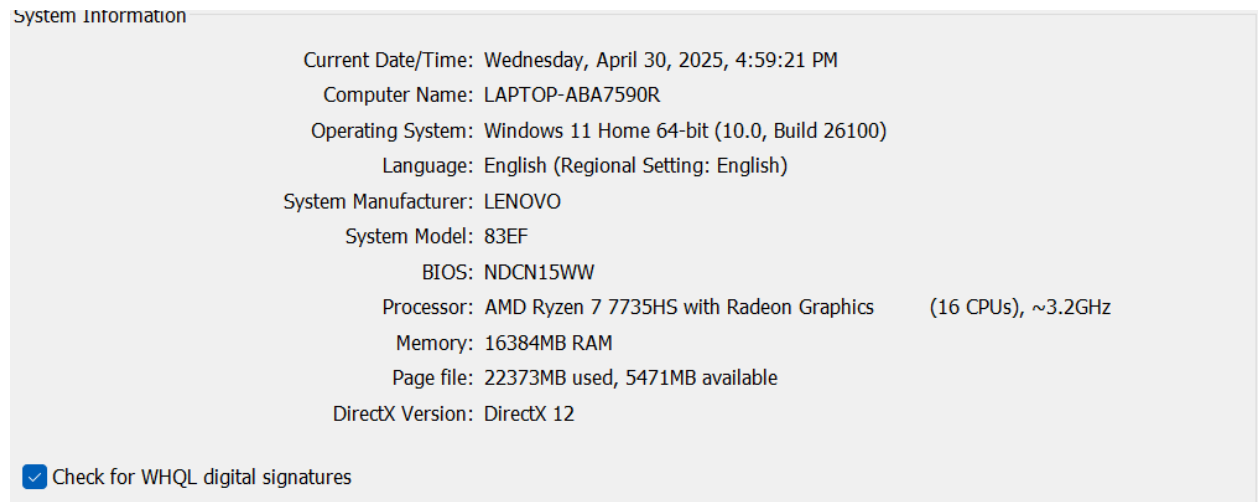
```

36 @Override
37 public int compareTo(Media other) {
38     if (other instanceof DigitalVideoDisc) {
39         DigitalVideoDisc d = (DigitalVideoDisc) other;
40
41         int titleCompare = this.getTitle().compareToIgnoreCase(d.getTitle());
42         if (titleCompare != 0) return titleCompare;
43
44         int lengthCompare = Integer.compare(d.getLength(), this.getLength());
45         if (lengthCompare != 0) return lengthCompare;
46
47         return Float.compare(this.getCost(), d.getCost());
48     }
49
50     // Nếu là media khác thì dùng logic mặc định (ví dụ: cost -> title)
51     int costCompare = Float.compare(this.getCost(), other.getCost());
52     if (costCompare != 0) return costCompare;
53
54     return this.getTitle().compareToIgnoreCase(other.getTitle());
55 }

```

6. String, StringBuilger and StringBuffer

So sánh thời gian xử lý xâu trên eclipse JAVASE-1.8 trên máy tính có cấu hình:

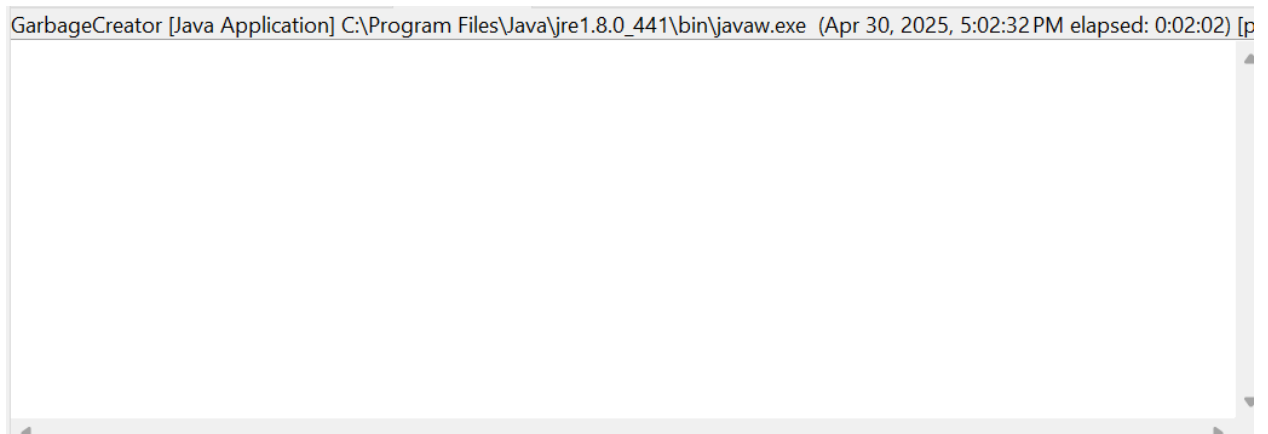


Ta có kết quả:

```
Time with String +: 252ms
Time with StringBuilder: 1ms
Time with StringBuffer: 1ms
```

Đánh giá:

- Khi chạy GarbageCreator, chương trình đã bị lag khi xử lý file text.txt có dung lượng 5MB.



- Trong khi đó, NoGarbage chạy mượt và cho ra kết quả:

Time to process with StringBuilder: 37ms

Time to process with StringBuffer: 65ms

- Kết luận:

	Thời gian chạy	Đánh giá
String +	Rất chậm	Hiệu suất kém nhất do tạo nhiều đối tượng rác
StringBuilder	37 ms	Hiệu suất tốt nhất nhưng không thread-safe
StringBuffer	65 ms	Hiệu suất tốt hơn String+ nhưng không bằng StringBuilder, thread-safe